CPSC 250
Big Program Two: Credit Card Debt Search Program

Edward Brash

1 Introduction

In this project, we consider a realistic programming task involving a large CSV data file.

Suppose we are given a large CSV file containing information about individuals in the United States, including:

- last names,
- states,
- credit card debt values.

Our goal is to write a Python program that can search this data efficiently and answer questions such as:

1. Find all people whose names start with a particular string.
2. Find all people with debt below some limit.
3. Find all people from a particular state.
4. Find the person with the highest debt.

Although this may initially seem like a large and intimidating task, the key idea is:

> Break the large problem into many smaller problems.

Each smaller problem is manageable and usually only requires a few lines of code.

2 Questions We Must Answer

Before writing code, we should think carefully about the design of the program.

Some important questions are:

1. What does the CSV file look like?
2. What data structures should we use?
3. What algorithms are needed for searching?

These are examples of software design questions.

3 Understanding the CSV File

Suppose our CSV file is called:

```
customer_data.csv
```

and contains rows such as:

```
Smith,VA,2450.75
Johnson,CA,10234.10
Jones,NY,500.00
Brown,TX,7890.50
```

Each row contains three pieces of information:

1. Last name,
2. State,
3. Credit card debt.

Notice that:

- names are strings,
- states are strings,
- debts are floating point numbers.

4 Choosing a Data Structure

The notes propose a very simple first design idea:

> Use three ordered lists.

Specifically:

```
names = []
states = []
debts = []
```

These are called **parallel lists**.

4.1 Why Parallel Lists Work

The important idea is that the entries at the same index correspond to the same person.

For example:

Index	names	states	debts
0	Smith	VA	2450.75
1	Johnson	CA	10234.10
2	Jones	NY	500.00
3	Brown	TX	7890.50

Thus:

```
names[2]
```

corresponds to:

```
states[2]
```

and:

```
debts[2]
```

because all three refer to the same customer.

5 Step 1: Read the Data from the File

The first major task is to read the CSV file and store the information in the lists.

The notes propose writing a function:

```
def read_customer_data(filename):
```

This function should:

1. open the file,
2. read every row,
3. extract the data,
4. store the data in the lists,
5. return the lists.

6 Opening a CSV File

Python provides built-in file handling tools.

A common pattern is:

```
with open(filename, "r") as file:
```

Important ideas:

- `"r"` means "read mode",
- `file` becomes a file object,
- the `with` statement automatically closes the file.

7 Using the CSV Module

Python includes a built-in module called `csv` for working with CSV files.

We begin by importing it:

```
import csv
```

Then we create a CSV reader:

```
reader = csv.reader(file)
```

The reader behaves like a loop over rows.
Each row is itself a list of strings.
For example, a row might look like:

```
["Smith", "VA", "2450.75"]
```

8 Reading Rows from the File

We loop through the rows:

```
for row in reader:
```

Inside the loop:

- `row[0]` is the name,
- `row[1]` is the state,
- `row[2]` is the debt.

We append these values to the appropriate lists.

9 The `append()` Method

Python lists provide the method:

```
my_list.append(value)
```

This adds a new value to the end of the list.
For example:

```
names.append("Smith")
```

adds the string `"Smith"` to the end of the list.

10 Converting Debt to a Float

When data is read from a CSV file, everything initially appears as a string.
For example:

```
row[2] == "2450.75"
```

But debt values should behave numerically.
Therefore we convert them:

```
float(row[2])
```

This converts the string into a floating point number.

11 Complete File Reading Function

Putting everything together:

```
import csv

def read_customer_data(filename):

    names = []
    states = []
    debts = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        for row in reader:

            names.append(row[0])
            states.append(row[1])
            debts.append(float(row[2]))

    return names, states, debts
```

12 Returning Multiple Values

Notice the line:

```
return names, states, debts
```

Python allows functions to return multiple values.
We can capture them using:

```
names, states, debts = read_customer_data("customer_data.csv")
```

This is extremely useful when working with related data structures.

13 Step 2: Get Information from the User

Next, the program asks the user for search information.
The notes suggest three kinds of input:

1. a debt limit,
2. a search phrase,
3. a state abbreviation.

For example:

```
debt_limit = float(input("Enter debt limit: "))
search_phrase = input("Enter search phrase: ")
st = input("Enter state abbreviation: ")
```

Notice:

- debt limits should be converted to numbers,
- names and states remain strings.

14 Searching for Names

Suppose we want all names beginning with:

```
"Jo"
```

We loop through the list of names.

```
for n in range(len(names)):
```

The expression:

```
range(len(names))
```

produces indices:

$$0, 1, 2, 3, \ldots$$

Using indices is important because we need access to the matching state and debt information.

15 The `startswith()` String Method

Python strings provide a method called:

```
startswith()
```

Example:

```
if names[n].startswith(search_phrase):
```

This checks whether the string begins with the desired text.
For example:

```
"Johnson".startswith("Jo")
```

returns:

```
True
```

while:

```
"Smith".startswith("Jo")
```

returns:

```
False
```

16 Complete Name Search Example

```
print("Matching names:")

for n in range(len(names)):

    if names[n].startswith(search_phrase):

        print(names[n], states[n], debts[n])
```

17 Searching by Debt

Suppose we want everyone with debt less than $2000.

We compare each debt value against the limit.

```
print("People below debt limit:")

for n in range(len(debts)):

    if debts[n] < debt_limit:

        print(names[n], states[n], debts[n])
```

Notice that this is a simple linear search through the data.

18 Searching by State

Suppose we want all customers from Virginia.

```
print("People from state:", st)

for n in range(len(states)):

    if states[n] == st:

        print(names[n], states[n], debts[n])
```

This uses the equality operator:

```
==
```

to compare strings.

19 Step 3: Find the Highest Credit Card Debt

Another important task is finding the maximum debt value.

The notes use a classic algorithm based on tracking the index of the largest value found so far.

20 Understanding the Maximum Search Algorithm

The algorithm is:

1. Assume the first value is the maximum.
2. Loop through the remaining values.
3. If a larger value is found, update the index.
4. At the end, the stored index corresponds to the largest debt.

21 Implementing the Maximum Search

```
index_max = 0

for n in range(len(debts)):

    if debts[n] > debts[index_max]:

        index_max = n
```

Important idea:

> We store the index of the maximum value, not the value itself.

This is useful because the index also gives access to the matching name and state.

22 Printing the Maximum Debt Customer

After the loop finishes:

```
print("Highest debt customer:")
print(names[index_max], states[index_max], debts[index_max])
```

Suppose:

Index	Name	State	Debt
0	Smith	VA	2450
1	Johnson	CA	10234
2	Jones	NY	500

Then:

```
index_max == 1
```

because Johnson has the largest debt.

23 Building Programs Incrementally

A very important lesson from this project is:

> Large programs should be built incrementally.

Instead of trying to write everything at once:

1. Write one small piece.
2. Test it.
3. Move to the next piece.
4. Test again.

This is how real software development works.

24 Program Decomposition

At first, a 100-line program can appear overwhelming.
However:

> Most large programs are really collections of many small programs.

In this project:

- reading the file is only a few lines,
- searching names is only a few lines,
- searching debts is only a few lines,
- finding a maximum is only a few lines.

The complexity comes from combining many simple ideas together.

25 Complete Example Program

```
import csv


def read_customer_data(filename):

    names = []
    states = []
    debts = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        for row in reader:

            names.append(row[0])
            states.append(row[1])
            debts.append(float(row[2]))

    return names, states, debts
```

```
# Read data from the CSV file
names, states, debts = read_customer_data("customer_data.csv")


# Get information from the user
debt_limit = float(input("Enter debt limit: "))
search_phrase = input("Enter search phrase: ")
st = input("Enter state abbreviation: ")


# Search for names
print()
print("Names beginning with", search_phrase)

for n in range(len(names)):

    if names[n].startswith(search_phrase):

        print(names[n], states[n], debts[n])


# Search by debt
print()
print("People below debt limit")

for n in range(len(debts)):

    if debts[n] < debt_limit:

        print(names[n], states[n], debts[n])


# Search by state
print()
print("People from", st)

for n in range(len(states)):

    if states[n] == st:

        print(names[n], states[n], debts[n])


# Find highest debt
index_max = 0

for n in range(len(debts)):

    if debts[n] > debts[index_max]:

        index_max = n


print()
print("Highest debt customer:")
```

```
print(names[index_max], states[index_max], debts[index_max])
```

26 Possible Improvements

This design works well for learning purposes, but there are many possible improvements.
For example:

- use dictionaries instead of parallel lists,
- use classes and objects,
- use pandas,
- sort the data,
- use binary search,
- add error checking,
- handle malformed CSV files,
- make searches case-insensitive.

However, the simple version is ideal for learning the fundamentals.

27 Computational Complexity

All searches in this program are linear searches.
That means:

$$\mathcal{O}(n)$$

time complexity.
Why?
Because in the worst case, we may need to examine every entry in the list.
For a file with:

10,000

rows, this is usually perfectly acceptable.
For millions of rows, more sophisticated data structures might become necessary.

28 Key Takeaways

- CSV files store structured tabular data.
- The `csv` module helps read CSV files safely.
- Parallel lists are a simple but useful data structure.
- File data is initially read as strings.

- Numerical values often require conversion using `float()` or `int()`.
- Linear searches are straightforward to implement.
- The `startswith()` method is useful for string searching.
- Maximum search algorithms often track indices rather than values.
- Large programs become manageable when broken into smaller pieces.